HK

Hell's Kitchen Delivery

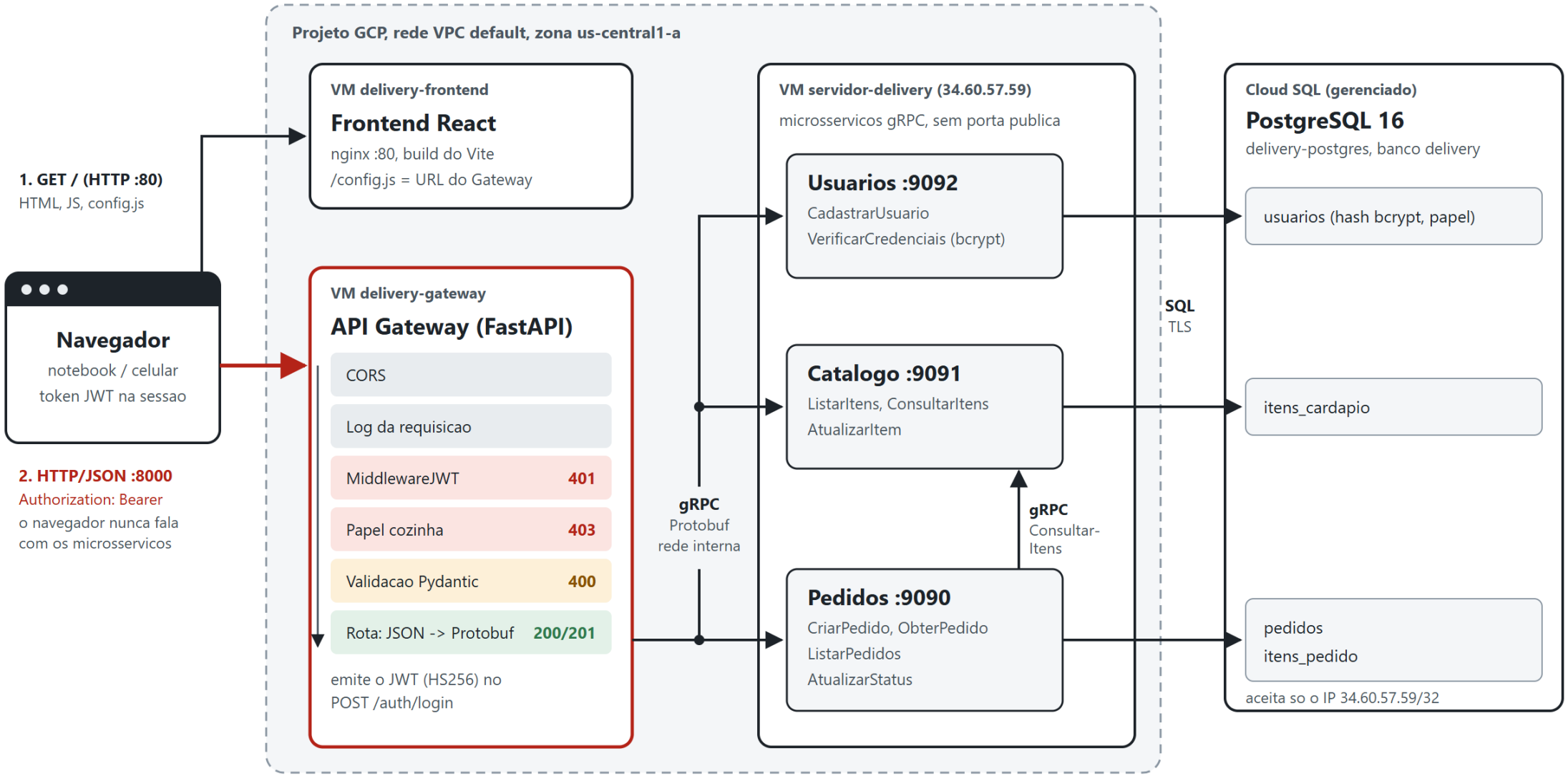
Frontend React, API Gateway FastAPI com JWT, três microsserviços gRPC e PostgreSQL no Google Cloud



Trabalho 2 de Sistemas Distribuídos, 2026/2
Matheus Maia, INF/UFG
github.com/matheus14maia/sd-2026-2

Hell's Kitchen Delivery: arquitetura de microsservicos no GCP

Frontend React -> API Gateway FastAPI (JWT, validacao, JSON -> Protobuf) -> 3 microsservicos gRPC -> PostgreSQL no Cloud SQL



Firewall: tcp:80 aberta para o frontend; tcp:8000 aberta para o Gateway; tcp:9090-9092 aceita so a VM com a tag delivery-gateway.

O navegador baixa a página da VM do frontend, mas todas as chamadas de dados vão direto ao Gateway. Os microsserviços não têm porta pública.

O caminho de um POST /pedidos

Cada etapa é uma camada diferente, em uma máquina diferente.

1 Navegador

Envia JSON com cliente, endereço e itens, e o cabeçalho Authorization: Bearer.

2 Gateway: borda

MiddlewareJWT confere assinatura e validade do token. Pydantic valida o payload.

3 Gateway: tradução

Monta CriarPedidoRequest (Protobuf binário) e chama PedidoService por gRPC.

4 Pedidos -> Catálogo

ConsultarItens por gRPC: o Catálogo diz se o item existe, se está disponível e o preço.

5 Pedidos -> Cloud SQL

Calcula o total e grava pedidos + itens_pedido numa única transação.

6 Volta ao navegador

Protobuf vira JSON no Gateway, que responde 201 Created com o total do servidor.

Gateway: o que é barrado na borda

Requisição inválida volta antes de qualquer chamada gRPC.

CORS

Log da requisição

MiddlewareJWT `401`

Papel cozinha `403`

Validação Pydantic `400`

Rota: JSON -> Protobuf -> gRPC `200/201`

Cenário	Status
Requisição sem token	401
Token com assinatura adulterada	401
Senha errada no login	401
Cliente tentando mudar preço	403
Pedido sem campo obrigatório	400
JSON malformado	400
Item inexistente (vem do microsserviço)	400
Item indisponível	409
Pedido válido	201

`src/gateway/autenticacao.py, esquemas.py, erros.py`

Contratos gRPC (.proto)

O .proto é a fonte da verdade. Os stubs Python são gerados no build da imagem.

usuarios.proto

service UsuarioService

```
rpc CadastrarUsuario
rpc VerificarCredenciais
```

Hash bcrypt; erro UNAUTHENTICATED vira 401.

catalogo.proto

service CatalogoService

```
rpc ListarItens
rpc ConsultarItens
rpc AtualizarItem
```

Dono do cardápio; consultado pelo Pedidos.

pedidos.proto

service PedidoService

```
rpc CriarPedido
rpc ObterPedido
rpc ListarPedidos
rpc AtualizarStatus
```

Chama o Catálogo por gRPC para precificar.

Erro do microsserviço (status gRPC) vira status HTTP no Gateway

INVALID_ARGUMENT	UNAUTHENTICATED	NOT_FOUND	FAILED_PRECONDITION	UNAVAILABLE
400	401	404	409	503

Dados reais no PostgreSQL

Cada microsserviço é dono das suas tabelas. Nenhuma lista em memória, nenhum mock.

Usuarios	`usuarios`	• INSERT no cadastro • SELECT no login (confere o hash)
Catalogo	`itens_cardapio`	• SELECT do cardápio e por categoria • UPDATE de preço e disponibilidade
Pedidos	`pedidos, itens_pedido`	• INSERT do pedido e itens numa transação • SELECT da lista e do detalhe • UPDATE condicional do status

4

tabelas no Cloud SQL, criadas pelo migrador (db/schema.sql)

Reiniciar os containers não perde nada: o estado está no banco.

SQL puro com psycopg 3 e pool de conexões, só nos arquivos repositorio.py. Senha só como hash bcrypt.

Deploy no Google Cloud

Três VMs e um Cloud SQL. Um único docker-compose.yml: cada máquina sobe o seu profile.

Máquina	Papel	Portas	Quem pode chegar
`delivery-frontend`	nginx com o build do React	`80`	internet
`delivery-gateway`	API Gateway FastAPI + JWT	`8000`	internet
`servidor-delivery`	Usuarios, Catalogo, Pedidos, migrador	`9090-9092`	só a VM do Gateway
`delivery-postgres`	Cloud SQL, PostgreSQL 16	`5432 (TLS)`	só o IP do servidor-delivery

Firewall por tag

As portas gRPC aceitam só a tag delivery-gateway.

DNS interno

O Gateway chama servidor-delivery pelo nome interno da VPC.

Um comando

scripts/gcp/ligar.ps1 liga banco e VMs e testa cada camada.

Demonstração ao vivo

Frontend

http://34.72.228.145

Gateway (Swagger)

http://34.132.172.125:8000/docs

Fluxo mostrado

login -> cardápio -> pedido 201 -> banco

Código

github.com/matheus14maia/sd-2026-2



Repositório no GitHub